

Algorithm Engineering für moderne Hardware

Ulrich Meyer · Volker Weichert

Received: date / Accepted: date

Zusammenfassung Die Entwicklung von effizienten Algorithmen basiert auf der theoretischen Modellierung der Rechner, auf denen sie ausgeführt werden sollen. Noch heute werden frühe Rechnermodelle wie die *Random Access Machine* (RAM) verwendet, um die Güte von Algorithmen zu analysieren und untereinander zu vergleichen. Aber sich verändernde Umstände wie der rasante Anstieg von interessanten Instanzgrößen, technologischer Fortschritt sowie neue Kostenmaße stellen neue Anforderungen an die Bewertung von Algorithmen – und damit auch an ihre Entwicklung.

In den letzten Jahren wurden viele Fortschritte hinsichtlich dieser neuen Herausforderungen gemacht. Seit den neunziger Jahren des vergangenen Jahrhunderts werden auf Basis des *I/O-Modells* Algorithmen entwickelt, die die Speicherhierarchien und -technologien moderner Rechner ausnutzen. Auch neueste Speichertechnologien wie Flash-Speicher wurden schon in einem Modell beschrieben, das es Entwicklern ermöglicht, effiziente Algorithmen dafür zu entwickeln.

Der andauernde Trend zu Mehrkernprozessoren führt dazu, dass viele Ergebnisse der klassischen Parallelverarbeitung auf praktisch allen modernen Rechnern Anwendung finden. Algorithmen, die nach dem *PEM-Modell* entwickelt

wurden können die grundlegenden Eigenschaften heutiger Prozessoren ausnutzen. Auch wenn es noch kein allgemein anerkanntes theoretisches Modell für Berechnungen auf Grafikprozessoren gibt, können Algorithmenentwickler hocheffiziente parallele Algorithmen für sie entwickeln, wenn sie sich der Eigenschaften der Hardware bewusst sind.

Wirtschaftliche Notwendigkeiten haben dazu geführt, dass sich ein zusätzliches Optimierungsziel für IT-Projekte etabliert hat – die Energieeffizienz. Auch das Algorithm Engineering kann seinen Teil dazu beitragen, zum Beispiel mit der Optimierung von Algorithmen für stromsparende Hardware.

Summary The development of efficient algorithms depends on good theoretical models of the target hardware. Early computer models like the *random access machine* (RAM) are still being used to analyze and compare algorithms today. But the circumstances are changing: the size of interesting instances is increasing rapidly, new technologies are developed and new measures for the performance of algorithms gain importance.

The last years have seen a lot of progress in respect to these challenges. Since the 1990s, algorithms making use of the memory hierarchy of modern computers have been developed using the *I/O-model*. Even models for new technologies like solid state disks have been created and thus enabled developers to write efficient algorithms.

There is an ongoing trend in hardware development to include more processor cores in a single CPU. Consequently, many results from classic parallel algorithmics can be applied to almost any modern computer. Algorithms that are developed using the *PEM-model* can make use of the characteristics of modern CPUs. Although there is no generally accepted theoretical model for graphics processors yet, algorithm designers can already create highly efficient parallel algorithms for them if they are aware of their specifics.

Partially supported by the DFG grant ME 3250/1-3, and by MADALGO – Center for Massive Data Algorithmics, a Center of the Danish National Research Foundation.

Ulrich Meyer
Institut für Informatik, Goethe Universität Frankfurt, Robert-Mayer-Str. 11–15, 60325 Frankfurt/Main
Tel.: +49-69-798-28433
Fax: +49-69-798-28357
E-Mail: umeyer@cs.uni-frankfurt.de

Volker Weichert
Institut für Informatik, Goethe Universität Frankfurt, Robert-Mayer-Str. 11–15, 60325 Frankfurt/Main
Tel.: +49-69-798-23155
Fax: +49-69-798-28814
E-Mail: weichert@cs.uni-frankfurt.de

Due to economic reasons, a new optimization target has been established for recent IT projects: energy efficiency. Algorithm engineering can further this goal with the design of algorithms optimized for hardware with low energy consumption.

1 Einführung

Weltweit wird gerade das 100. Geburtsjahr von Alan Turing (1912–1954), einem der Begründer der Informatik begangen. Mit der *Turing Maschine* [13] hat er ein klassisches Berechnungsmodell aufgestellt, das bis heute benutzt wird um die Klasse der mit einem Computerprogramm effizient lösbar Probleme zu charakterisieren. Natürlich hat die Turing Maschine – grob gesprochen ein programmierbarer Kassettenrekorder – auf den ersten Blick nur noch geringe Ähnlichkeit mit einem aktuellen Computer. Aber auch modernere Konzepte, wie das RAM Modell mit adressierbarem Speicher (statt eines Bandes wie bei der Turing Maschine) weisen eine stetig wachsende Diskrepanz zu aktueller Hardware auf. Werden Algorithmen auf der Basis dieser klassischen Berechnungsmodelle entwickelt, besteht die latente Gefahr, die resultierenden Programmlaufzeiten signifikant falsch einzuschätzen und/oder das Potential moderner Hardware nicht auszuschöpfen. In diesem Artikel skizzieren wir einige der algorithmischen Herausforderungen, die sich durch Speicherhierarchien, Parallelverarbeitung und die Einbeziehung des Energieverbrauchs ergeben.

2 Speichereffizienz

Während der Wartezeit für einen Speicherzugriff können moderne Prozessoren 1000 Registeroperationen durchführen; für Festplattenzugriffe kann dieser Faktor sogar auf mehrere Millionen ansteigen. Im klassischen Algorithmenentwurf hingegen, werden für alle Operationen und Speicherzugriffe identische Kosten angesetzt – mit oftmals verheerenden Folgen für die Programmlaufzeiten, sobald die Daten nicht mehr in den Hauptspeicher passen.

In der Tat besitzen moderne Rechner eine ganze Hierarchie von Speichern (Register, L1, L2, Hauptspeicher, Festplatten, Backup, etc). Um ein Datum verarbeiten zu können, muss sich dieses in einem Register befinden. Falls nicht (Cache-Miss), wird es von seinem aktuellen Speicherort durch alle dazwischenliegenden Speicherebenen zum Register transportiert. Diese Speichertransfers erfolgen immer blockweise, in der Hoffnung, dass benachbarte Datenelemente in unmittelbarer Zukunft ebenfalls benötigt werden. Diese Annahme ist jedoch in den seltensten Fällen automatisch erfüllt. Es ist deshalb oft unabdingbar, die Anzahl der Cache-Misses von Anfang an durch geeignete Datenstrukturierung

und cache-effiziente Algorithmen zu minimieren um damit den Datendurchsatz signifikant zu erhöhen.

Schon ein kleines Beispiel [12] demonstriert das Potential optimierter Speicherzugriffe: Das Transponieren von Matrizen ist eine fundamentale Operation in der linearen Algebra und in schnellen Fourier-Transformationen. Der einfachste C++ Code zum Transponieren einer $N \times N$ Matrix (Loop) sieht wie folgt aus:

```
for (i = 0; i < N; i++)
  for (j = i+1; j < N; j++)
    swap(A[i][j], A[j][i])
```

Leider können aufgrund der zeilenorientierten Speicherung der Matrix bei der Ausführung obigen Loop-Codes bis zu N^2 Cache-Misses auftreten, indem für jeden Elementaustausch eine „neue“ (d.h. nicht im Cache vorhandene) Matrixzeile angefasst werden muss. Deutlich bessere Performance wird durch einen *Divide & Conquer* Algorithmus erzielt, der das Problem in zwei Teil-Matrizen aufspaltet (d.h. an der „längeren“ Seite der Matrix mittig teilt), und diese rekursiv transponiert. Je nach Rechner werden am Rekursionsende ein einziger oder konstant viele Matrizeneinträge transponiert. Für die Datentransfers zwischen zwei benachbarten Speicherebenen mit Blockgröße B liegt die Zahl der Cache-Misses nun in der Größenordnung von N^2/B statt N^2 . Es ist klar, dass dies für große Werte von B (z.B. $B \simeq 10^5$ für den Übergang zwischen Hauptspeicher und Festplatten) rentabel ist, aber selbst bei einer reinen Internspeicher-Hierarchie (mit typischerweise sehr kleinen Werten für B) ist der kompliziertere Divide & Conquer Algorithmus bis zu 10 mal schneller, siehe Abb. 1.

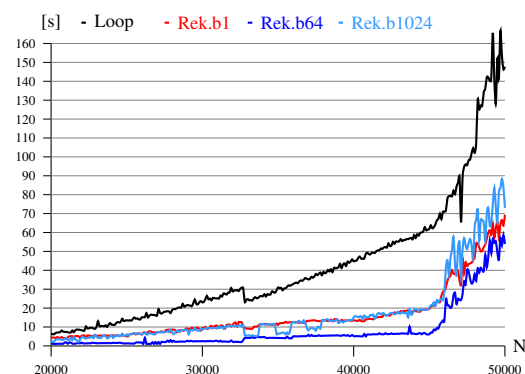


Abb. 1 Schnelleres Transponieren einer Matrix durch optimierte Speicherzugriffe: Laufzeitvergleich zwischen Loop-Algorithmus und rekursiven Lösung mit verschiedenen Abbruchgrößen. Gut erkennbar ist auch der Übergang zwischen Hauptspeicher und Swap-space ab $N \approx 45.000$.

2.1 Cache-Awareness und Cache-Obliviousness

Generell ist es erstrebenswert, Algorithmen derart zu gestalten, dass sie sich variablen Nebenbedingungen im Speicherumfeld (z.B. geänderte Block- oder Hauptspeichergrößen durch neue Hardware) auch über mehrere Hierarchieebenen hinweg automatisch anpassen (Cache-Obliviousness [9]). Wie im Falle des Matrizentransponierens geschieht dies zumeist durch rekursive Algorithmen und Datenstrukturen. Leider ergeben sich dabei oft schlechtere Laufzeitkonstanten als bei Ansätzen, die speziell für zwei übereinanderliegende Speicherebenen mit bekannter Blockgröße optimiert wurden (Cache-Awareness).

2.2 I/O-Modell in der Praxis

Um im Kontext von Speicherhierarchien vernünftige Laufzeitvorhersagen zu erzielen braucht es Berechnungsmodelle, die die Wirklichkeit hinreichend abbilden aber gleichzeitig bezüglich ihrer Komplexität noch handhabbar bleiben. Das einfachste und gleichsam erfolgreichste Modell für den Datenaustausch zwischen zwei Speicherebenen (Intern- und Extern-Speicher) ist das sogenannte I/O-Modell [1], bei dem lediglich die Parameter M und B vorkommen: M bezeichnet die Größe des begrenzten Internspeichers, der Datentransfer zwischen Intern- und Extern-Speicher erfolgt in Blöcken der Größe B . Optimierungsgröße ist (neben der klassischen Anzahl der Berechnungsschritte) gerade die Anzahl dieser Datentransfers (I/Os). Vitter listet in einem Übersichtsartikel [17] mit mehr als 350 Referenzen eine Vielzahl von Datenstrukturen und Algorithmen für das I/O-Modell. Im Algorithm Engineering wird untersucht, ob sich diese zumeist theoretischen Resultate in praktisch messbare Leistungssteigerungen übersetzen. Das ist gerade beim einfachen I/O-Modell und klassischen Festplatten à priori überhaupt nicht klar: äußere Plattenspuren liefern größere Transferaten als innere Spuren, physikalisch benachbarte Blöcken sind u.U. deutlich schneller nacheinander zu lesen als weiter entfernt liegende Blöcke, etc. Es zeigt sich jedoch, dass diese Effekte für hinreichend große Blöcke (im Megabyte-Bereich) regelmäßig in den Hintergrund treten und das I/O-Modell ein durchaus brauchbares Werkzeug darstellt. Nichtkommerzielle Software-Bibliotheken, die einige Datenstrukturen und Algorithmen für das I/O-Modell zur Verfügung stellen, sind TPIE [4] und STXXL [8]. Auf kommerzieller Ebene vertreibt beispielsweise das Spin-off Unternehmen Scalgo ¹ I/O-effiziente Software-Lösungen im Bereich Geländemodellierung und Hochwasservorhersage.

¹ <http://scalgo.com/technology/ioefficiency.php>

2.3 Neue Speichermedien

Mit jeder neuen Speichertechnologie ist jedoch eine Überprüfung bisheriger Modellannahmen angebracht. Bei *Flashspeicher* (Solid State Disks, SSDs), der in den letzten Jahren nicht nur in Digitalkameras und Smartphones sondern auch in Notebooks und Hochleistungsservern immer größere Verbreitung findet, müssen zwei verschiedene Blockgrößen unterschieden werden: eine kleinere, B_r , zum Lesen und eine zum Teil deutlich größere, B_w , zum Löschen bzw. Überschreiben von Daten auf dem Flash-Speicher. Zusätzlich ist die Anzahl der Schreibzyklen pro Flashspeicherzelle limitiert, so dass komplizierte *Wear-leveling* Mechanismen eingesetzt werden müssen, damit sich die Zellen nicht einseitig und vorschnell abnutzen. Die Leistungsfähigkeit des Flashspeichers ist damit sowohl von den benutzten Controllern als auch der (Schreib-) Zugriffshistorie abhängig [6].

Mit dem sog. *Phase Change Memory* (PCM) steht schon die nächste (vermutlich bald massentaugliche) Speichertechnologie in der Tür: im Gegensatz zu Flashspeicher müssen zum Schreiben einzelner Daten keine größeren Bereiche mehr gelöscht werden und auch die Anzahl der möglichen Schreibzyklen liegt deutlich höher. Dennoch wird eine Diskrepanz zwischen Lesen und Schreiben bestehen bleiben, die sich in größerem Energieverbrauch, höherer Latenz, sowie geringerer Bandbreite beim Schreiben bemerkbar macht. Algorithmische Gegenmaßnahmen durch geeignete Datenstrukturen werden z.B. in [7] erläutert.

3 Parallelität

In den letzten Jahren haben sich die Fähigkeiten einfacher Computer, Probleme parallel zu berechnen stark weiter entwickelt. Moderne CPUs haben inzwischen bis zu 16 Prozessorkerne, Tendenz steigend. Moderne Grafikkarten bieten die Möglichkeit, ihre auf parallele Verarbeitung von tausenden Pixeln optimierte Hardware für beliebige Berechnungen zu benutzen. Es gibt Mobiltelefone mit Mehrkernprozessoren und Spielekonsolen mit Cell-Technologie. Die massive Verbreitung dieser Komponenten macht die Parallelisierung von Programmen in allen Bereichen, von spezieller Steuerungs- und Planungssoftware bis zum Officepaket interessant. Die Besonderheiten der unterschiedlichen Komponenten sind Augenmerk des Algorithm Engineering.

3.1 Multicore-CPUs

Bei *Multicore-CPUs* sitzen mehrere Prozessorkerne auf einem Chip. Sie teilen sich die meisten Ressourcen des Computers, zum Beispiel Massenspeicher, Ein- und Ausgabegeräte und Hauptspeicher, verfügen aber auch über eigene, private Komponenten. Algorithmisch besonders interes-

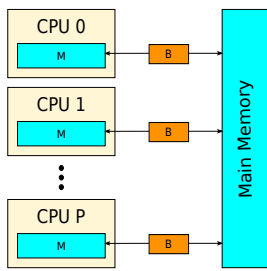


Abb. 2 Schema des PEM Modells

sant sind hierbei L1- und L2-Caches von mehreren Kilobyte Größe. Der gleichberechtigte Zugriff auf manche Stufen der Speicherhierarchie, vor allem des Hauptspeichers, legt dabei nahe, ein solches System als *Shared Memory Multicomputer* zu betrachten und algorithmisch mithilfe von PRAMs zu modellieren. Allerdings haben PRAMs nur konstant viel lokalen Speicher pro Prozessoreinheit und haben das aus Abschnitt 2 bekannte Problem, dass ein Speicherzugriff und eine Berechnung dieselben Kosten verursachen. Dieses Modell spiegelt die Wirklichkeit daher nur unzureichend wieder.

Um der Architektur gerecht zu werden haben Arge et al. in [3] das *Parallel External Memory* (PEM) Modell entwickelt. Das Modell geht davon aus, dass sämtliche Kommunikation zwischen Prozessorkernen über gemeinsame Stufen der Speicherhierarchie stattfindet und die Geschwindigkeitsunterschiede zwischen den Stufen der Speicherhierarchie analog zum Externspeichermodell [1] ausschlaggebend für die Gesamtlaufzeit der Algorithmen sind.

Das PEM-Modell (vgl. Abb. 2) ist eine Verallgemeinerung des I/O-Modells, in dem statt einem Prozessor nun P Prozessoren, jeweils mit privatem Cache der Größe M , gleichzeitig Zugriff auf den großen Hauptspeicher haben. Wie im I/O-Modell werden die Daten in Blöcken der Größe B transportiert. Parallele Zugriffe auf unterschiedliche Blöcke des Hauptspeichers sind erlaubt, für gleichzeitige Zugriffe auf dieselben Blöcke wird wie bei PRAMs zwischen *Concurrent Read Concurrent Write* (CRCW), *Concurrent Read Exclusive Write* (CREW) und *Exclusive Read Exclusive Write* (EREW) unterschieden. Auch hier soll die Anzahl der Datenübertragungen minimiert werden.

Algorithmus 1 Addition von n Zahlen im PEM-Modell

```

procedure SUMME( $A[0 : N - 1]$ )
   $B = \emptyset$ 
  for all Prozessor  $i$  in parallel do
    Berechne Summe  $s$  von  $A[i \cdot N/P : (i + 1) \cdot N/P - 1]$ 
     $B[i] = s$ 
  end for
  Prozessor 0 lädt  $B$  und berechnet seine Summe  $s_t$ 
  return  $s_t$ 
end procedure

```

Während die üblichen Probleme der Parallelisierung auf dem Level der theoretischen Modellierung bewältigt werden können, zeigen sich manche Details erst bei der Implementierung. Die Kommunikation der Prozessoren über die gemeinsamen Stufen der Speicherhierarchie birgt Probleme: *Race conditions* entstehen wenn die Ausgabe von Subroutine s_1 auf Prozessor p_i die Eingabe für Subroutine s_2 auf Prozessor $p_j, i \neq j$ ist und p_j die Eingabe lesen will, bevor p_i die entsprechende Ausgabe geschrieben hat. Die Lösung für solche Probleme ist meistens eine Synchronisation. Da Synchronisation aber nur über Kommunikation zwischen allen beteiligten Prozessoren erfolgen kann, ist sie sehr kostspielig und sollte deshalb möglichst selten verwendet werden.

Es gibt inzwischen viele Frameworks für die Programmierung von Shared Memory Systemen wie Multicore-CPU's, zum Beispiel OpenMP² und OpenHMPP³. Des Weiteren ist auch die Message Passing Programmierung mit MPI⁴ für Multicore-CPU's möglich. In diesem Fall wird ein Distributed Memory System simuliert und jeder Prozessor verwaltet seinen eigenen Speicherbereich, wobei das Übertragen der Nachrichten hier durch effiziente Kommunikationsfunktionen im gemeinsamen Speicher übernommen wird. Damit ist ohne Weiteres eine Skalierung jenseits eines einzelnen Multicore-Computers möglich.

Bei Verwendung eines Message Passing Protokolls können race conditions leicht vermieden werden. Das geschieht allerdings zu Lasten des gemeinsamen Zugriffs auf dieselben Daten im Hauptspeicher und kann zur Vervielfachung des Speicherbedarfs führen, da jeder Prozessor gemeinsame Daten für sich selbst im Speicher ablegen muss. Die Ausführungsgeschwindigkeit des fertigen Programms ist dabei stark vom verwendeten Framework abhängig. Erwartungsgemäß erzielt OpenMP im Allgemeinen höhere Speedups als MPI, das aber trotzdem gute Werte erreicht [14].

3.2 GPGPU

Das große Interesse an *General Purpose Computation on Graphics Processor Units* (GPGPU) wurde 2007 mit der Veröffentlichung von NVIDIAs⁵ *Compute Unified Device Architecture* (CUDA) geweckt. CUDA erlaubte Entwicklern erstmals, Programme und Datenstrukturen auf Grafikkarten zu implementieren, ohne sich dabei in das enge Korsett der Grafik-Pipeline zwingen zu müssen. In den Jahren seitdem gab es einen ständigen Fluss von Veröffentlichungen von Algorithmen für Grafikprozessoren (GPUs), wobei teilweise beeindruckende Speedups von mehreren Hundert gegenüber existierenden Algorithmen für CPUs erzielt wurden (zum Beispiel [16], [18]).

² <http://www.openmp.org>

³ <http://www.openhmpp.org/>

⁴ <http://www.mcs.anl.gov/mpl>

⁵ <http://www.nvidia.com>

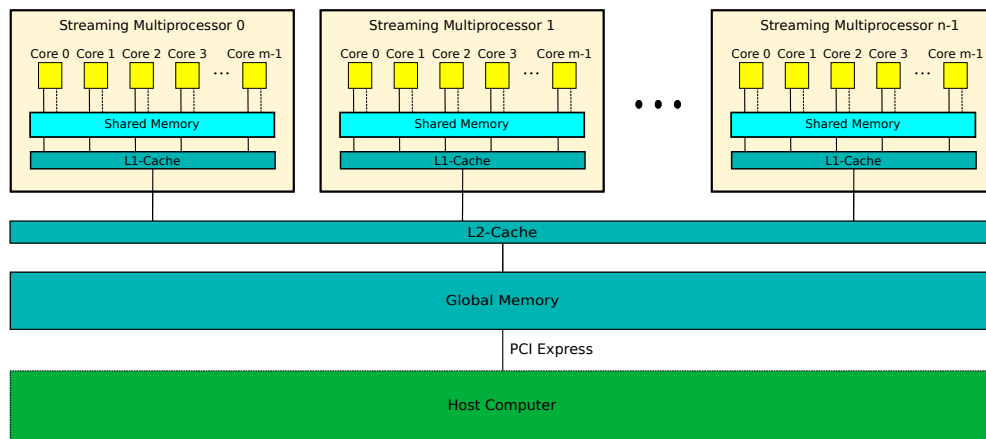


Abb. 3 Schematische Darstellung einer Fermi-GPU mit Grafikspeicher und Anbindung an den Gastrechner

Die Entwicklung von Algorithmen für Grafikkarten muss sich stark am Hardwarelayout orientieren. Auf einer modernen GPU befinden sich mehrere Prozessoren, die jeweils mit mehreren Kernen ausgestattet sind und über ihren eigenen Speicher verfügen (siehe Abb. 3). Außerdem hat jeder Prozessor Zugriff auf den Grafikspeicher, der nicht auf der GPU selbst sitzt und somit eine hohe Latenz aufweist. Im Vergleich zu den privaten Caches der Prozessoren ist er aber sehr groß und die einzige Möglichkeit der Prozessoren untereinander oder mit dem Rest des Computers zu kommunizieren. Aufgrund dieser Ähnlichkeiten können viele Konzepte von Algorithmen für Multicore-CPU's übernommen werden. Manche Details bedingen aber nicht-triviale Veränderungen und manche Veränderungen sind einfach nötig um die Hardware optimal auszunutzen. Im Folgenden werden wir auf NVIDIAs Fermi-Architektur und CUDA eingehen [15].

Ein CUDA Programm, auch *Kernel* genannt, wird in b Blöcken zu je t Threads ausgeführt. Zur Vereinfachung der Addressierung können die Blöcke dreidimensional sein und zweidimensional angeordnet werden, aber auf die Programmausführung hat das keinen Einfluss. Die Anzahl der Threads pro Block oder der Blöcke selbst ist hierbei nicht von der Anzahl der Kerne oder Prozessoren beschränkt. Im Gegenteil, eine höhere Anzahl erlaubt den Prozessoren die Maskierung von Speicherlatenzen, indem Wartezeiten mit Arbeit anderer Threads überbrückt werden.

Die tatsächliche Ausführung geschieht in Gruppen zu 32 Threads, eine Einheit die sich *Warp* nennt. Die Threads eines Warps führen jeden Schritt gemeinsam aus. Sollten innerhalb eines Warps datenbedingte Verzweigungen auftreten, zum Beispiel durch ein IF...THEN...ELSE..., werden die unterschiedlichen Programmpfade von den jeweiligen Threads nacheinander ausgeführt. Im Extremfall könnte der gesamte Warp serialisiert werden. Um also optimale Performance zu erreichen muss der Kernel so ausgelegt sein,

dass zumindest innerhalb eines Warps alle Threads dem selben Programmpfad folgen.

Eine weitere Besonderheit der GPUs sind die verschiedenen Speicherarten, die alle Vor- und Nachteile haben. Der größte, aber auch langsamste Speicherbereich ist der schon erwähnte globale Speicher. Er befindet sich physikalisch nicht auf der GPU und ist mehrere Gigabytes groß. Es können besondere Bereiche des globalen Speichers als Texturen- oder Konstantenspeicher definiert werden. Diese Bereiche sind dann read-only, können also nur vom Hostcomputer befüllt werden. Allerdings verfügen die einzelnen Prozessoren für diese Bereiche über eigene Caches, was bei mehrfach benötigten, konstanten Daten durchaus ein Vorteil sein kann. Der globale Speicher selbst wird für alle Prozessoren durch einen 768 kB großen L2-Cache gepuffert. Jeder Prozessor verfügt über einen 64 kB großen privaten Cache, der zwischen dem *Shared Memory* und dem L1-Cache für globale Speicherzugriffe geteilt wird und 32684 64-Bit-Register. Über alle Speicherbereiche außer den L1- und L2-Caches hat der Programmierer die volle Kontrolle.

Die auszuführenden Thread-Blöcke werden gleichverteilt den Prozessoren zugeteilt. Jeder Prozessor lädt so viele von den ihm zugeteilten Blöcken wie möglich. Die limitierenden Ressourcen sind hier die Anzahl der Register, die Menge an Shared Memory und die Gesamtzahl der Threads, die nicht mehr als 1536 pro Prozessor sein darf. Vor allem bei Kernels, die den Shared Memory benutzen um Ein- oder Ausgabedaten zu puffern ist das eine wichtige Stellschraube.

Während alle Threads eines Blocks über den Shared Memory Daten austauschen können gibt es keine Möglichkeit für Blöcke, direkt untereinander zu kommunizieren. Solche Kommunikation muss immer über den globalen Speicher passieren. Um dabei race conditions zu vermeiden wird Synchronisation benötigt, die bei GPUs noch kostspieliger ist als bei Multicore-CPU's. Die meistbenutzte Methode ist, den Kernel zu beenden und einen neuen Kernel zu starten, der

dann mit den Daten weiterarbeitet, die der vorherige Kernel im globalen Speicher zurück ließ. Allerdings haben Daten im Shared Memory nur die Reichweite eines Kernels, weswegen für eine globale Synchronisation alle Zwischenergebnisse aus dem Shared Memory im globalen Speicher zwischengelagert oder neu berechnet werden müssen.

Das Konzept der Datenlokalität hat sich auch bei der Entwicklung von Algorithmen für Grafikkarten als sehr wichtig erwiesen. Parallele Zugriffe auf Daten, die konsekutiv im Grafikspeicher stehen, werden zu größeren Datenübertragungen zusammengefasst. Es ist also sinnvoll, die Daten so zu strukturieren, dass die Threads eines Blocks auch auf entsprechend zusammenhängende Blöcke an Ein- und Ausgabedaten zugreifen. Dieses Konzept nennt sich *coalesced memory accesses*.

Etwas ähnliches gilt für den Shared Memory. Dieser ist in 32 Bänken zu je zwei Kilobyte aufgeteilt. Die einzelnen Einträge eines Datenfeldes auf diesem Speicher werden automatisch zirkulierend auf die unterschiedlichen Bänke abgebildet. Da die Ausführung des Kernels warpweise geschieht wird also von jeder Speicherbank ein Datum gelesen, wenn die Threads konsekutive Einträge des Feldes anfordern. Sollten mehrere Threads eines Warps jedoch unterschiedliche Daten anfordern, die auf der gleichen Speicherbank lagern liegt ein *bank conflict* vor und die Anfragen werden serialisiert und nacheinander beantwortet. Für den Fall, dass mehrere Threads das gleiche Datum anfordern existiert eine Broadcastfunktion.

Gerade die Probleme bei der globalen Kommunikation und Synchronisation wiegen schwer bei den meisten Problemen. Dennoch gibt es viele Anwendungen, bei denen GPGPU durch die parallele Verarbeitung von tausenden Threads einen entscheidenden Vorteil bieten kann, wobei Speedups von durchaus mehreren Hundert gemessen werden können. Meistens handelt es sich hierbei um trivial parallelisierbare Probleme, aber auch in vielen komplexeren Bereichen konnten sehr gute Ergebnisse erzielt werden.

4 Energieeffizienz

Seit wenigen Jahren ist der Begriff *Green Computing* in der IT-Branche allgegenwärtig, nicht ohne Grund: Laut einer aktuellen Studie [10] betrug der Jahresstromverbrauch der Server und Rechenzentren in Deutschland in 2011 insgesamt 9,7 Terawattstunden (TWh). Das entspricht immerhin in etwa der Jahresproduktion von vier mittleren Kohlekraftwerken. Zwar sind die Server an sich sparsamer geworden und benötigen weniger Kühlung, die Energiepreise jedoch weiter gestiegen, so dass sich für die Rechenzentren zwischen 2008 und 2012 die Stromkosten um 12 Prozent erhöht haben. Welchen Beitrag kann die Informatik auf der Algorithm Engineering Seite leisten?

Zunächst gilt recht grob, dass eine schnellere Programmausführung auf der gleichen Maschine tendenziell weniger Energie benötigt und deshalb effiziente Programme mehr oder minder automatisch den Energieverbrauch reduzieren. Bei genauerer Betrachtung ist das Verhältnis zwischen Ausführungszeit und Energieverbrauch komplizierter. In Kapitel 2 haben wir schon erwähnt, dass bei neuen Speichermedien Schreiboperationen deutlich mehr Energie verbrauchen als Leseoperationen. Somit kann ein etwas langsamerer Algorithmus, der Teilergebnisse neu berechnet statt sie abzuspeichern, durchaus energieeffizienter sein - bei unveränderter Hardwarekonfiguration.

4.1 Power-Down Mechanismen

Können Teile der Hardware nicht nur manuell sondern auch vom Betriebssystem oder Anwenderprogramm bei Bedarf deaktiviert werden, ergeben sich weitere algorithmisch nutzbare Einsparpotentiale: In Phasen *längerer* Inaktivität sollte die entsprechende Komponente möglichst schnell abgeschaltet werden, damit sie nicht weiter unnötig Strom verbraucht. Bei *kurzen* Inaktivitätsphasen ist das Abschalten jedoch kontraproduktiv, da eine Wiederinbetriebnahme i.A. mit höherem Energiebedarf (und oftmals auch Zeitverzögerungen) einhergeht. Die Länge der nächsten Inaktivitätsphase ist dem Betriebssystem/Anwenderprogramm leider meistens nicht bekannt; wir sprechen dann von einem *Online-Problem*. Der Qualitätsmaßstab für Online-Algorithmen ist der sog. *Wettbewerbsfaktor*, auch *Kompetitivität* genannt, bei dem die (Energie-) Kosten $A(\sigma)$ eines Online Algorithmus A auf einer Inaktivitätssequenz σ mit den Kosten $OPT(\sigma)$ eines optimalen Offline-Algorithmus (der die gesamte Sequenz σ schon kennt) verglichen werden. A ist c -kompetitiv falls $A(\sigma)/OPT(\sigma) \leq c$ für jede Inaktivitätssequenz σ gilt. Indem ein Online-Algorithmus A mit dem Abschalten einer Komponente wartet, bis deren Weiterbetrieb gerade soviel Energie verbraucht hat wie das spätere Wiedereinschalten benötigen wird, ist A schon 2-kompetitiv. Interessanterweise ist *jeder deterministische* Online-Algorithmus für dieses Problem im Worst-Case nicht besser als 2-kompetitiv, es existiert jedoch eine *randomisierte* Lösung mit erwartetem Wettbewerbsfaktor $e/(e-1) \approx 1.58$, wobei $e \approx 2.71$ für die Eulersche Zahl steht. Außerdem wurden Erweiterungen für Komponenten mit mehr als zwei Betriebszuständen entworfen. Mehr Details zu diesen Ansätzen finden sich zum Beispiel in Übersichtsartikeln von Albers [2] oder Irani und Pruhs [11], sowie den darin enthaltenen Referenzen.

4.2 Dynamic Speed Scaling

Eine weitere Möglichkeit, den Energieverbrauch der Datenverarbeitung zu drücken liegt in der sog. *Dynamic Speed*

Scaling Technik: moderne Prozessoren lassen sich oft mit unterschiedlichen Geschwindigkeiten betreiben – der Energieverbrauch steigt jedoch typischerweise super-linear (z.B. kubisch) mit der Geschwindigkeit an. Deshalb wird insbesondere bei Mehrprozessorsystemen versucht, die zu bearbeitenden Aufgaben so auf die verfügbaren Prozessoren mit entsprechenden Geschwindigkeiten zu verteilen, dass die Gesamtenergie minimiert wird, ohne gewisse Leistungsgarantien zu opfern. Das ist schon auf nur einem einzigen Prozessor mit sequentieller oder quasi-paralleler Ausführung ein nicht-triviales Scheduling-Problem, bei dem nun nicht nur die Abarbeitungsreihenfolge (und möglicherweise auch Unterbrechungen), sondern auch jeweiligen Geschwindigkeiten passend zu wählen sind. Je nach Ausgestaltung der Optimierungsziele und Nebenbedingungen (Deadlines, Antwortzeiten, Energie- oder Temperatur-Maxima, ...) ergeben sich somit interessante Offline- und Online-Scheduling Probleme. Eine vertiefte algorithmische Behandlung des Dynamic Speed Scaling findet man wiederum in den oben bereits erwähnten Übersichtsartikeln [2, 11].

4.3 Hardware-Software Co-Design

Ein abschließendes Beispiel für eine Algorithm Engineering Arbeit, die gleich mehrere der in diesem Papier angesprochenen Aspekte (Speicherhierarchien, Parallelität und Energie) beinhaltet, ist das EcoSort⁶ Projekt [5]. Ziel dieser Kooperation zwischen KIT Karlsruhe und Goethe Universität Frankfurt war es, eine Hardware/Software Kombination zu erstellen, die möglichst energieeffizient große Datenmengen sortieren kann. Dazu gibt es mit *Joule Sort* als Teil des *Sort Benchmark*⁷ einen internationalen Wettbewerb mit klar definierten Regeln, welche Datensätze zu sortieren sind, wie der Energieverbrauch gemessen wird etc. Insbesondere zählt *nur* der Energieverbrauch, nicht die Sortierzeit – somit kann ein langsamerer Ansatz auf einer sparsamen Hardware besser sein als ein schnellerer auf einer energiehungrigen Maschine.

Für Ecosort wurde statt eines traditionellen Servers ein stromsparendes Board mit Intel Atom 330 Prozessor und vier Consumer-Level 256 GB SSDs als Datenspeicher sowie einem USB-Stick für das Betriebssystem benutzt (siehe Abb. 4). Auf der Softwareseite kamen speziell angepasste I/O-effiziente Mergesort-Algorithmen mit Multi-Core Unterstützung zum Einsatz. Datentransfer und interne Sortierschritte wurden überlagert, um die Hardwarekomponenten möglichst gleichmäßig auszulasten. Weiterhin konnte bei kleineren Instanzen ein Teil des Hauptspeichers deaktiviert und auf größeren Eingaben durch Inplace-Sortierverfahren der

Bedarf teuren SSD-Speicherplatzes in überschaubarem Rahmen gehalten werden.

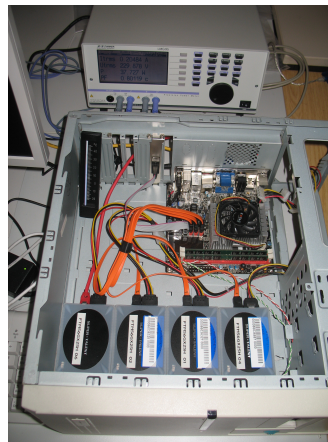


Abb. 4 Ecosort Rechner mit Energie-Meßgerät / Logo des Bundeswettbewerbs.

In der Summe wurden 2010 durch Ecosort mehrere bestehende Rekorde der Joule Sort Kategorie um Faktoren drei bis fünf verbessert und teilweise auch 2011 noch gehalten. Das Projekt wurde weiterhin als *ausgewählter Ort* in der Bundesinitiative *Deutschland – Land der Ideen*⁸ 2011 ausgezeichnet. Erst 2012 hat ein Team der Intel Labs Pittsburgh und Carnegie Mellon Universität die letzten Ecosort Rekorde überflügelt – mit einer deutlich kostspieligeren Hardware aus Intel Core i7-2700K 3.5 GHz Prozessor und 16 Hochleistungs-SSDs (Intel 710 Series 300GB) – jede einzelne dieser SSDs kostet derzeit in Deutschland über 1000 Euro, während das Ecosort Gesamtsystem in 2010 mit weniger als 3000 Euro zu Buche schlug.

Natürlich werden sich im Alltagsbetrieb derartige Hardwareinvestitionen kaum durch entsprechende Energieeinsparungen amortisieren – ein gewichtetes Kostenmaß aus Anschaffungs- und Nutzungskosten könnte realistischer sein. Andererseits würde zum Beispiel die Mehrheit aller Smartphone Benutzer vermutlich überproportional tiefer in die Tasche greifen wollen, um die Laufzeit ihrer geliebten mobilen Begleiter signifikant zu verlängern. Die Erforschung energieeffizienter Hardware-Software Kombinationen sollte uns also höchstwahrscheinlich weiter begleiten.

Literatur

1. A. Aggarwal and J. S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
2. S. Albers. Energy-Efficient Algorithms. *Communications of the ACM*, 53(5):86–96, 2010.

⁶ <http://www.ecosort.de>

⁷ <http://sortbenchmark.org/>

⁸ <http://www.land-der-ideen.de>

3. L. Arge, M. T. Goodrich, M. Nelson and N. Sitchinava. Fundamental parallel algorithms for private-cache chip multiprocessors.
4. L. Arge, O. Procopiuc, and J. S. Vitter. Implementing I/O-Efficient Data Structures Using TPIE. In *Proc. 10th European Symposium on Algorithms (ESA)*, volume 2461 of LNCS, pages 88–100, Springer, 2002. Library Download under <http://www.madalgo.au.dk/tpie/>
5. A. Beckmann, U. Meyer, P. Sanders, and J. Singler. Energy-Efficient Sorting using Solid State Disks. *Sustainable Computing: Informatics and Systems* (Elsevier), 1(2):151–163, June 2011.
6. L. Bouganim, B. Jónsson, and P. Bonnet. uFLIP: Understanding Flash IO Patterns. In *Proc. 4th Conference on Innovative Data Systems Research*, 2009.
7. S. Chen, P. B. Gibbons, and S. Nath. Rethinking Database Algorithms for Phase Change Memory. In *Proc. 5th Conf. on Innovative Data Systems Research (CIDR)*, 2011.
8. R. Dementiev, L. Kettner, and P. Sanders. STXXL: Standard Template Library for XXL data sets. *Software Practice & Experience*, 38(6):589–637, 2008. Library available under <http://stxxl.sourceforge.net>
9. M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran. Cache-oblivious algorithms. In *40th FOCS*, pages 285–297. IEEE, 1999.
10. R. Hintemann and K. Fichter. Energieverbrauch und Energiekosten von Servern und Rechenzentren in Deutschland – Aktuelle Trends und Einsparpotenziale bis 2015. *Borderstep Institut für Innovation und Nachhaltigkeit, Berlin*, 2012.
11. S. Irani, and K. Pruhs. Algorithmic problems in power management. *SIGACT News* 36, pages 63–76, 2005.
12. P. Kumar. Cache Oblivious Algorithms. In *Algorithms for Memory Hierarchies*, volume 2625 of LNCS, pages 193–212. Springer, 2003.
13. H. Lewis and C. Papadimitriou. Elements of the Theory of Computation, second edition. Prentice-Hall 1997.
14. D. Mallón, G. Taboada, C. Teijeiro, J. Touriño, B. Fraguera, A. Gómez, R. Doallo, J. Mourino, M. Ropo, J. Westerholm and J. Dongarra. Performance Evaluation of MPI, UPC and OpenMP on Multicore Architectures. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, volume 5759 of LNCS, pages 174–184. Springer, 2009.
15. NVIDIA Corporation CUDA C Programming Guide 4.2 NVIDIA, 2012.
16. C. Richardt, D. Orr, I. Davies, A. Criminisi and N. A. Dodgson. Real-time Spatiotemporal Stereo Matching Using the Dual-Cross-Bilateral Grid. In *Proceedings of the European Conference on Computer Vision (ECCV)*, volume 6313 of LNCS, pages 510–523. Springer, 2010.
17. J. S. Vitter. *Algorithms and Data Structures for External Memory*. Foundations and Trends in Theoretical Computer Science. now Publishers, 2008.
18. Y. Zhao, X. Chen, C.-W. Sham, W. Tam and F. Lau. Efficient Decoding of QC-LDPC Codes Using GPUs. In *Algorithms and Architectures for Parallel Processing*, volume 7016 of LNCS, pages 294–305. Springer, 2011.